Java Backend Architecture

Interview Series

Chapter 17.4
Ingress & TLS Termination

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 17.4 - Ingress and TLS Termination

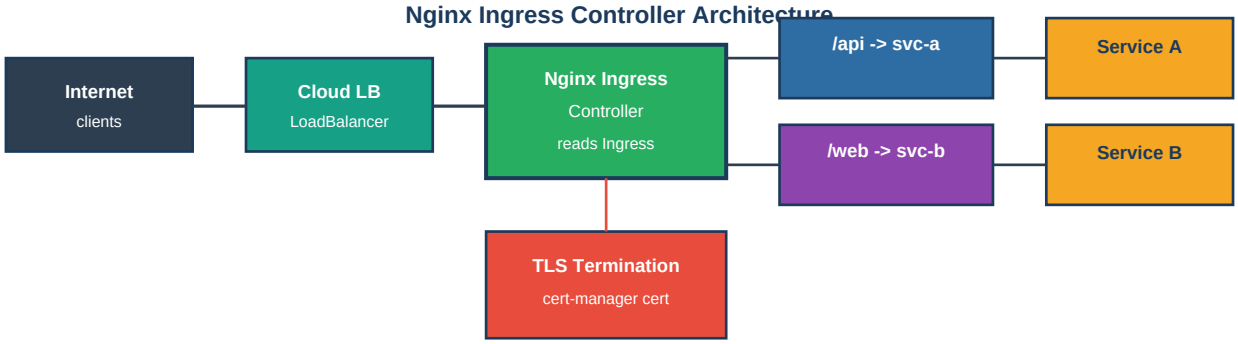
Overview

Ingress provides HTTP/HTTPS routing at L7, enabling host-based and path-based routing, TLS termination, and rewrite rules. The Nginx Ingress Controller is the most widely used implementation. cert-manager automates TLS certificate provisioning from Let's Encrypt. Gateway API is the next-generation replacement for Ingress.

Key Definitions

Ingress	K8s API object defining HTTP routing rules: host + path -> Service backend.
Ingress Controller	Pod that implements Ingress rules (Nginx, Traefik, HAProxy, AWS ALB, GCP GLB).
Nginx Ingress	Popular Ingress controller; generates nginx.conf from Ingress objects; Deployment + LoadBalancer Svc.
cert-manager	K8s add-on; automates TLS cert provisioning from ACME (Let's Encrypt), Vault, or self-signed.
ClusterIssuer	cert-manager cluster-scoped issuer; references ACME server, private key Secret.
Certificate	cert-manager CRD; requests a cert for given DNS names; results in K8s TLS Secret.
TLS passthrough	Forward raw TLS to backend without termination; app handles TLS itself.
Gateway API	Next-gen K8s routing: GatewayClass, Gateway, HTTPRoute, GRPCRoute, TCPRoute.
HTTPRoute	Gateway API object defining HTTP routing rules; type-safe, role-oriented design.
pathType	Exact (exact match), Prefix (prefix match), ImplementationSpecific (controller-dependent).

Diagram 1: Nginx Ingress Controller Architecture



Ingress reads Secret with TLS cert | nginx.conf auto-generated | HTTPS terminated at Nginx

Diagram 2: cert-manager ACME Flow

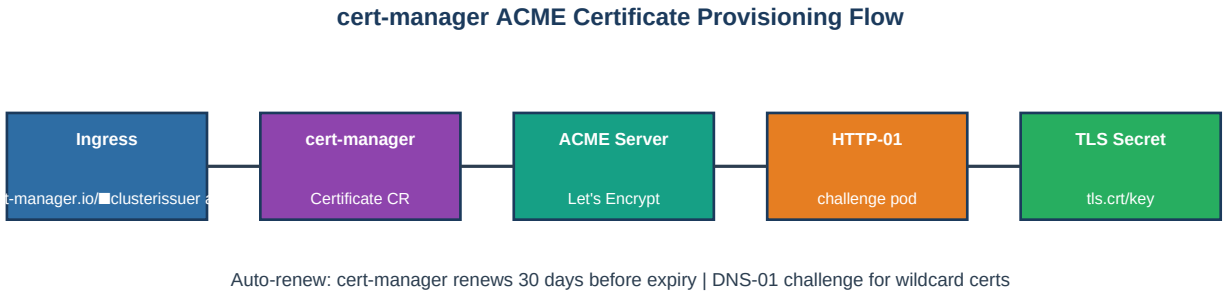
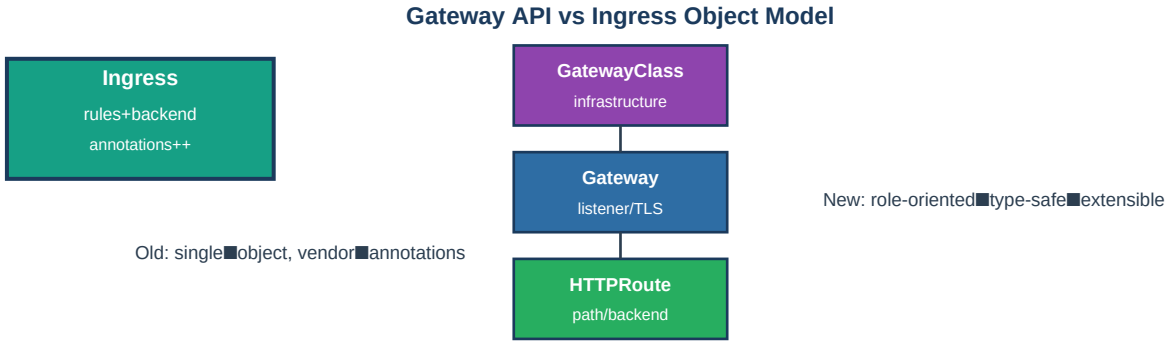


Diagram 3: Gateway API vs Ingress



Code Examples

Host-based and path-based Ingress:

```
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: java-api-ingress annotations: nginx.ingress.kubernetes.io/rewrite-target: /$2 nginx.ingress.kubernetes.io/proxy-body-size: 10m nginx.ingress.kubernetes.io/limit-rps: "100" cert-manager.io/cluster-issuer: letsencrypt-prod spec: ingressClassName: nginx tls: - hosts: [api.example.com] secretName: api-tls rules: - host: api.example.com http: paths: - path: /api(/|$)(.*) pathType: Prefix backend: service: name: java-api port: {number: 80} - path: /health pathType: Exact backend: service: name: health-svc port: {number: 8080}
```

cert-manager ClusterIssuer:

```
apiVersion: cert-manager.io/v1 kind: ClusterIssuer metadata: name: letsencrypt-prod spec: acme: server: https://acme-v02.api.letsencrypt.org/directory email: admin@example.com privateKeySecretRef: name: letsencrypt-prod-key solvers: - http01: ingress: class: nginx - dns01: route53: region: us-east-1 # for wildcard certs
```

Gateway API HTTPRoute:

```
apiVersion: gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name: java-api-route spec: parentRefs: - name: main-gateway hostnames: [api.example.com] rules: - matches: - path: type: PathPrefix value: /api backendRefs: - name: java-api port: 80 weight: 90 - name: java-api-canary port: 80 weight: 10 filters: - type: RequestHeaderModifier requestHeaderModifier: add: - name: X-Forwarded-Proto value: https
```

Interview Q&A; - Ingress and TLS

Q1. What is a Kubernetes Ingress and how does it differ from a Service?

Ingress is an L7 HTTP/HTTPS routing object that can route based on hostname and URL path, perform TLS termination, and rewrite URLs - all from a single external IP (one LoadBalancer Service). Service LoadBalancer: one external IP per Service, L4 only (TCP/UDP). Ingress: one external IP for many Services, L7 routing, TLS termination, annotations for auth/rate-limiting.

Q2. How does the Nginx Ingress Controller work?

Nginx Ingress Controller runs as a Deployment with a LoadBalancer Service. It uses an informer to watch Ingress objects in all namespaces. When Ingress objects change, it generates nginx.conf with server blocks (virtual hosts), upstream blocks (service endpoints), and location blocks (paths). Applies config via nginx -s reload. Supports SSL termination, rate limiting, auth, rewrites via annotations.

Q3. Explain path types in Ingress: Exact, Prefix, ImplementationSpecific.

Exact: path must match exactly - /api matches only /api, not /api/users. Prefix: path matches if request path has the prefix - /api matches /api, /api/users, /api/v1/. ImplementationSpecific: behavior defined by Ingress controller - Nginx supports regex in path. Best practice: use Exact for specific endpoints, Prefix for service hierarchies. Order matters: more specific paths should be listed first.

Q4. How does cert-manager automate TLS certificate provisioning?

cert-manager watches Ingress objects for cert-manager.io/cluster-issuer annotation. It creates a Certificate CR for the specified hosts. For HTTP-01 challenge: creates temporary Ingress route /.well-known/acme-challenge/ and a pod serving the challenge token. ACME server verifies and issues cert. cert-manager stores cert in K8s Secret (tls.crt, tls.key). Auto-renews 30 days before expiry.

Q5. What is the difference between HTTP-01 and DNS-01 ACME challenges?

HTTP-01: ACME server makes HTTP request to domain/.well-known/acme-challenge/token. Requires domain to be publicly accessible on port 80. Cannot issue wildcard certs. DNS-01: cert-manager creates TXT record _acme-challenge.domain.com in DNS. ACME server verifies TXT record. Works for private clusters, supports wildcard certs (*.domain.com). Requires DNS provider integration (Route53, Cloudflare, etc.).

Q6. What are common Nginx Ingress annotations?

nginx.ingress.kubernetes.io/rewrite-target: /$2 - rewrite URL path using capture groups. nginx.ingress.kubernetes.io/limit-rps: 100 - rate limit per IP. nginx.ingress.kubernetes.io/auth-url - external auth service (OAuth2 proxy). nginx.ingress.kubernetes.io/proxy-body-size: 50m - max request body (default 1m). nginx.ingress.kubernetes.io/ssl-redirect: "true" - redirect HTTP to HTTPS. nginx.ingress.kubernetes.io/affinity: cookie - sticky sessions.

Q7. How does TLS termination work in Nginx Ingress?

Ingress spec.tls references a Secret of type kubernetes.io/tls containing tls.crt and tls.key. Nginx Ingress reads the Secret and configures SSL virtual host. HTTPS traffic is decrypted at Nginx, forwarded as HTTP to Service (within cluster). Nginx uses SNI (Server Name Indication) to select correct certificate for multi-host Ingress. cert-manager populates the Secret automatically from ACME/Vault.

Q8. What is the Gateway API and why was it created?

Gateway API is the next-generation replacement for Ingress, addressing its limitations: Ingress is too HTTP-only (no TCP/UDP/gRPC natively), too dependent on vendor annotations, no role separation (infrastructure vs app teams). Gateway API: GatewayClass (infrastructure, managed by platform team), Gateway (deployment, cluster-ops team), HTTPRoute/GRPCRoute (app routing, dev team). Type-safe, extensible, role-oriented.

Q9. Explain TLS passthrough in Nginx Ingress.

TLS passthrough: Nginx forwards raw TLS bytes to backend without decrypting. Backend application handles TLS termination (e.g., databases requiring client cert auth). Configure: nginx.ingress.kubernetes.io/ssl-passthrough: "true". Nginx uses SNI to route to correct backend (no full TLS handshake at Nginx). Limitation: cannot inspect or modify HTTP headers (traffic is encrypted to Nginx). Use for databases (PostgreSQL with TLS), gRPC with mTLS.

Q10. How do you configure basic authentication in Nginx Ingress?

Create htpasswd Secret: kubectl create secret generic basic-auth --from-file=auth. Annotations: nginx.ingress.kubernetes.io/auth-type: basic, nginx.ingress.kubernetes.io/auth-secret: basic-auth, nginx.ingress.kubernetes.io/auth-realm: "Authentication Required". For OAuth2: use nginx.ingress.kubernetes.io/auth-url pointing to oauth2-proxy Service. auth-url validates JWT/session; on success, original request proceeds.

Q11. How does the Ingress controller discover Ingress objects?

Nginx Ingress Controller uses K8s informers to watch Ingress objects across all namespaces (or specific namespace via --watch-namespace). Uses SharedIndexInformer backed by K8s watch API. On Ingress add/update/delete event: controller enqueues reconciliation. Reconcile loop: collects all Ingress objects, builds nginx.conf, applies via nginx -s reload. ingressClassName field selects which controller handles each Ingress.

Q12. What is an IngressClass?

IngressClass is a cluster-scoped resource identifying which controller handles Ingress objects. spec.controller: k8s.io/ingress-nginx for Nginx controller. Ingress spec.ingressClassName: nginx selects the IngressClass. Multiple controllers can run simultaneously (Nginx for internal, ALB for external). Default IngressClass: ingressclass.kubernetes.io/is-default-class: "true" annotation - applied to Ingresses without explicit ingressClassName.

Q13. How do you implement canary deployments with Nginx Ingress?

Create second Deployment (canary) and Service. Create canary Ingress with annotations: nginx.ingress.kubernetes.io/canary: "true". nginx.ingress.kubernetes.io/canary-weight: 20 (20% of traffic). nginx.ingress.kubernetes.io/canary-by-header: X-Canary (header-based routing). Nginx routes the configured percentage to canary backend. Gradually increase weight to roll out new version.

Q14. What is the proxy-connect-timeout and proxy-read-timeout?

proxy-connect-timeout: time to connect to backend pod (default 5s). proxy-read-timeout: time to wait for response from backend (default 60s). proxy-send-timeout: time to send request to backend. For slow Java endpoints (long-running reports, complex queries): nginx.ingress.kubernetes.io/proxy-read-timeout: 300 (5 minutes). Combine with server.tomcat.connection-timeout in Spring Boot.

Q15. How does cert-manager handle certificate renewal?

cert-manager controller monitors Certificate CRs. Checks expiry: if within renewBefore period (default 30 days before expiry), triggers renewal. Creates new CertificateRequest and ACME Order, completes challenge, updates Secret. Pod does not need restart for cert renewal (nginx hot-reloads updated cert). Monitor: kubectl get certificates -A and status.conditions for Ready/False.

Q16. What are Gateway API routing features not available in Ingress?

Header-based routing (route by X-Version header). Query parameter matching. Weight-based traffic splitting (canary in spec, not annotations). Request/response header modification. URL redirect with code. Multiple parent gateways per route. gRPC-specific routing via GRPCRoute. TCP/TLS routing via

TCPRoute/TLSRoute. Cross-namespace routing (parent/child namespaces).

Q17. How do you configure CORS in Nginx Ingress?

nginx.ingress.kubernetes.io/enable-cors: "true" nginx.ingress.kubernetes.io/cors-allow-origin: "https://app.example.com" nginx.ingress.kubernetes.io/cors-allow-methods: "GET, POST, OPTIONS" nginx.ingress.kubernetes.io/cors-allow-headers: "Authorization, Content-Type" nginx.ingress.kubernetes.io/cors-max-age: "3600" For Spring Boot: disable application-level CORS if Nginx handles it.

Q18. What is the default backend in Nginx Ingress?

Default backend: handles requests that do not match any Ingress rule. Returns 404 or custom error page. Configure via --default-backend-service flag or Ingress spec.defaultBackend (Ingress-level fallback). Custom error pages: nginx.ingress.kubernetes.io/custom-http-errors: 404,503 with a custom error page Service that serves branded error pages. Used for consistent error responses across all services.

Q19. How do you force HTTPS redirect with Nginx Ingress?

Global: nginx.ingress.kubernetes.io/ssl-redirect: "true" (default when TLS configured). Force HTTPS even without TLS configured: nginx.ingress.kubernetes.io/force-ssl-redirect: "true". Nginx returns 308 Permanent Redirect from HTTP to HTTPS. For Spring Boot behind proxy: server.forward-headers-strategy=framework ensures accurate scheme detection from X-Forwarded-Proto header.

Q20. What is the difference between Ingress and API Gateway?

Ingress: K8s-native L7 routing; good for simple host/path routing and TLS. API Gateway (Kong, Ambassador, AWS API GW): advanced features: JWT auth, rate limiting per API key, request transformation, API versioning, developer portal, monetization, analytics dashboard. For Java microservices: use Ingress for internal routing; API Gateway for external API management with authentication, rate limiting, and developer access control.

Q21. How does Nginx Ingress handle WebSockets?

WebSocket upgrade requires: nginx.ingress.kubernetes.io/proxy-read-timeout: 3600 (keep connection open). Nginx automatically handles Upgrade header. Configuration: nginx.ingress.kubernetes.io/configuration-snippet adds custom nginx directives. For Spring WebSocket (STOMP over WebSocket): ensure connection is sticky (nginx affinity: cookie) if using in-memory session state.

Q22. What is a Certificate resource in cert-manager?

Certificate CRD defines desired TLS cert: spec.dnsNames, spec.issuerRef (ClusterIssuer or Issuer), spec.secretName (K8s Secret to create), spec.duration (cert lifetime, default 90 days), spec.renewBefore (renew when within 30 days of expiry). Cert-manager creates CertificateRequest -> ACME Order -> completes challenge -> stores cert in Secret. Status.conditions shows Ready state.

Q23. How do you debug a 503 error from Nginx Ingress?

kubectl logs -n ingress-nginx deployment/ingress-nginx-controller: look for upstream errors. kubectl get endpoints svc-name: verify pods are Ready and have IPs. kubectl describe ingress: check rules and backend service name/port match. Test direct Service access from inside cluster: kubectl exec pod -- curl svc-name:port. Check pod readinessProbe - 503 often means no Ready endpoints.

Q24. What is the nginx.ingress.kubernetes.io/rewrite-target annotation?

Rewrites the URL path before forwarding to backend. Example: Ingress path /api(/|$)(.*) with rewrite-target: /$2 strips /api prefix: request /api/users -> backend receives /users. Without rewrite: backend receives full path /api/users. Use capture groups $1,$2 from regex path. Alternative:

nginx.ingress.kubernetes.io/use-regex: "true" enables regex in path.

Q25. Explain Vault PKI integration with cert-manager.

cert-manager Vault Issuer: spec.vault.server, spec.vault.path, spec.vault.auth.kubernetes (uses ServiceAccount token for Vault K8s auth). Vault PKI secrets engine: generates short-lived certificates signed by internal CA. No public ACME required - works for internal services and private domains. Vault rotates signing CA automatically. cert-manager renews certs before expiry. Used for internal mTLS between Java microservices.

Q26. How does Nginx Ingress handle gRPC routing?

Nginx supports gRPC over HTTP/2. Annotations: nginx.ingress.kubernetes.io/backend-protocol: "GRPC" (for TLS-terminated gRPC) or "GRPCS" (passthrough). nginx.ingress.kubernetes.io/grpc-backend: "true". TLS must be configured on Ingress (gRPC requires HTTP/2 which requires HTTPS). Backend service: type ClusterIP, targetPort matching gRPC server port.

Q27. What is the difference between Nginx Community and Nginx Plus Ingress?

Nginx Community (nginx/nginx-ingress): open source, nginx-based, uses ConfigMap + annotations. Nginx Plus (nginxinc/kubernetes-ingress): commercial, adds advanced LB algorithms, active health checks, session persistence, JWT auth, App Protect WAF. Kubernetes official Nginx Ingress (kubernetes/ingress-nginx): most widely used, community-maintained, uses nginx.conf template approach. Most teams use kubernetes/ingress-nginx.

Q28. How do you configure rate limiting in Nginx Ingress?

nginx.ingress.kubernetes.io/limit-rps: 10 - 10 requests/second per IP. nginx.ingress.kubernetes.io/limit-rpm: 100 - 100 requests/minute per IP. nginx.ingress.kubernetes.io/limit-connections: 5 - max concurrent connections per IP. nginx.ingress.kubernetes.io/limit-whitelist: "10.0.0.0/8" - exempt IPs from limiting. Rate limiting uses nginx limit_req_zone with burst allowance.

Q29. What is ModSecurity/WAF integration with Nginx Ingress?

Nginx Ingress supports ModSecurity WAF (Web Application Firewall). Enable: nginx.ingress.kubernetes.io/enable-modsecurity: "true". OWASP CRS: nginx.ingress.kubernetes.io/enable-owasp-core-rules: "true". Blocks common attacks: SQL injection, XSS, CSRF, path traversal. For Java Spring apps: complement with Spring Security but WAF adds network-level protection. Alert-only mode: modsecurity-snippet with SecRuleEngine DetectionOnly.

Q30. How does the AWS ALB Ingress Controller work?

AWS Load Balancer Controller watches Ingress objects with ingressClassName: alb. Creates AWS Application Load Balancer with: listener (HTTP/HTTPS), target groups (one per Service backend), routing rules (path/host conditions). Annotations: alb.ingress.kubernetes.io/scheme: internet-facing, alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:... (ACM cert), alb.ingress.kubernetes.io/target-type: ip (pod IPs registered directly).

Q31. What is SNI routing in Nginx Ingress?

SNI (Server Name Indication): TLS extension that includes hostname in ClientHello. Nginx uses SNI to select correct certificate for multi-host TLS Ingress. Each TLS host entry in Ingress has its own Secret with certificate. Nginx can serve different certs to api.example.com and www.example.com from same IP. Default certificate (--default-ssl-certificate) used when SNI does not match any configured host.

Q32. How do you implement IP whitelisting with Nginx Ingress?

nginx.ingress.kubernetes.io/whitelist-source-range: "10.0.0.0/8,192.168.0.0/16" Restricts access to specified CIDR ranges. Returns 403 for other IPs. Combined with externalTrafficPolicy: Local to preserve client IP. For

admin endpoints (/actuator, /admin): add IP whitelist annotation. For country-based blocking: use nginx-geoip-module or Cloudflare WAF.

Q33. What is the difference between Ingress path routing and Service mesh routing?

Ingress path routing: L7 HTTP routing at cluster edge (north-south traffic, external to internal). Service mesh (Istio VirtualService): L7 routing for internal traffic (east-west, service to service). Service mesh can do weight-based routing, header routing, fault injection between internal services. Both can work together: Ingress for external entry, service mesh for internal policies.

Q34. How do you monitor Nginx Ingress performance?

Nginx Ingress exposes Prometheus metrics at :10254/metrics. Key metrics: nginx_ingress_controller_requests (request count by status), nginx_ingress_controller_request_duration_seconds (latency histograms), nginx_ingress_controller_nginx_process_connections (active connections). Grafana dashboard ID 9614 for Nginx Ingress. Alert on: 5xx rate > 1%, p99 latency > 1s, nginx worker CPU saturation.

Q35. What is the proxy-next-upstream configuration?

nginx.ingress.kubernetes.io/proxy-next-upstream: "error timeout" Retries request on next upstream pod if current fails. proxy-next-upstream-tries: 3 - max retry attempts. proxy-next-upstream-timeout: 10s - max time for retries. For idempotent GET requests: safe to retry. For POST requests: NOT safe unless endpoint is idempotent (risk of duplicate operations). Disable for mutation endpoints.

Q36. How does cert-manager use Kubernetes RBAC?

cert-manager creates ClusterRoles and ClusterRoleBindings for its ServiceAccounts. cert-manager controller: needs access to CertificateRequest, Certificate, Order, Challenge CRDs, read/write Secrets (for cert storage and private keys), Ingress (to read cert-manager annotations). cert-manager webhook: needs to be registered as ValidatingWebhookConfiguration. Check: kubectl auth can-i create certificates.cert-manager.io -n default --as=system:serviceaccount:cert-manager:cert-manager.

Q37. What is a Gateway in Gateway API?

Gateway is an instance of a GatewayClass, defining infrastructure for receiving traffic. spec.gatewayClassName references a GatewayClass (which controller to use). spec.listeners: define ports, protocols (HTTP/HTTPS/TCP), TLS configuration. spec.addresses: request specific external IP. Routes (HTTPRoute, TCPRoute) attach to Gateway via spec.parentRefs. Platform team manages Gateway; app teams manage Routes.

Q38. How do you implement OAuth2 authentication with Nginx Ingress?

Deploy oauth2-proxy (Bitnami/oauth2-proxy): Service that validates OAuth2 sessions. Annotations: nginx.ingress.kubernetes.io/auth-url: https://auth.example.com/oauth2/auth nginx.ingress.kubernetes.io/auth-signin: https://auth.example.com/oauth2/start nginx.ingress.kubernetes.io/auth-response-headers: Authorization,X-Auth-User oauth2-proxy handles redirect to IdP (Google, Okta, Keycloak), validates token, returns 200 or 401. Nginx proceeds or returns 401.

Q39. What is load balancing algorithm in Nginx Ingress?

Default: round-robin. Configure via: nginx.ingress.kubernetes.io/load-balance: "ewma" (exponential weighted moving average - routes to least latency pod), "random two least conn" (random selection from two least loaded). upstream-hash-by: sticky routing by request property (cookie, IP, header). For Java session affinity: use cookie-based affinity or stateless JWT approach.

Q40. How does the Ingress controller handle large file uploads?

Default nginx client_max_body_size: 1MB. For file uploads: nginx.ingress.kubernetes.io/proxy-body-size: 100m (100MB max). Set 0 for unlimited (use with caution - risk of disk DoS). proxy-request-buffering: off - stream large uploads directly to backend without buffering. Spring Boot: spring.servlet.multipart.max-file-size=100MB and max-request-size=100MB. Consider: direct upload to S3/GCS instead of routing through Nginx.

Q41. What is mutual TLS at the Ingress level?

mTLS Ingress: requires client to present certificate. nginx.ingress.kubernetes.io/auth-tls-secret: namespace/ca-secret (CA cert to verify client certs). nginx.ingress.kubernetes.io/auth-tls-verify-client: "on" (require) or "optional" (allow). nginx.ingress.kubernetes.io/auth-tls-pass-certificate-to-upstream: "true" (forward cert to app). Use case: API clients with machine certificates, B2B integrations.

Q42. How do you handle Ingress for multiple environments (dev/staging/prod)?

Option 1: separate clusters per environment (Ingress rules isolated). Option 2: separate namespaces with separate Ingress controllers or IngressClass. Option 3: namespace-scoped Ingress with different host names (api.dev.example.com, api.staging.example.com, api.example.com). Helm values.yaml: override host and TLS secret per environment. cert-manager: separate ClusterIssuer for staging (Let's Encrypt staging CA for testing).

Q43. What is the nginx.ingress.kubernetes.io/configuration-snippet annotation?

Allows inserting raw nginx directives into the location block. Example: add_header X-Frame-Options "SAMEORIGIN"; or proxy_hide_header X-Powered-By; Use with caution: can override security settings. Global snippets: nginx.ingress.kubernetes.io/server-snippet (server block), main-snippet via ConfigMap for global nginx.conf additions. Security: restrict ConfigMap and Ingress write access (RBAC) to prevent snippet injection.

Q44. How does Nginx Ingress handle HTTP/2?

Nginx Ingress enables HTTP/2 when TLS is configured (HTTP/2 over TLS, ALPN h2 negotiation). HTTP/2 features used: header compression (HPACK), multiplexing (multiple requests per connection). Backend connection: Nginx -> Service uses HTTP/1.1 by default (keepalive). For gRPC backend: proxy_http_version 1.1 + grpc_pass directive. To force HTTP/2 to backend: use GRPC annotation.

Q45. What are the limitations of Kubernetes Ingress?

HTTP/HTTPS only (no TCP/UDP natively - workarounds needed). Annotations are implementation-specific (not portable across controllers). No native weight-based routing (each controller implements differently). Limited role separation (single object for both infra and app routing). No status feedback on routing (only .loadBalancer.ingress). Gateway API addresses all these limitations with typed resources.

Q46. How does Nginx Ingress implement session persistence (affinity)?

Cookie-based affinity: nginx.ingress.kubernetes.io/affinity: cookie nginx.ingress.kubernetes.io/session-cookie-name: INGRESSCOOKIE nginx.ingress.kubernetes.io/session-cookie-expires: 172800 (2 days) nginx.ingress.kubernetes.io/session-cookie-path: / Nginx sets cookie on first response; subsequent requests with same cookie go to same pod. Pod must have stable identity (StatefulSet) or shared session store (Redis).

Q47. What is the cert-manager ingress-shim?

ingress-shim watches Ingress objects with cert-manager.io/cluster-issuer annotation. Automatically creates Certificate CRs for each TLS host. When Certificate becomes Ready, the referenced Secret (tls.secretName) is populated. Nginx Ingress reads the Secret for TLS configuration. No manual Certificate CR creation needed - annotation on Ingress is sufficient. Certificate CRs are owned by Ingress (garbage collected when Ingress

deleted).

Q48. How do you configure connection draining in Nginx Ingress?

When pod is terminating, kube-proxy removes it from endpoints. Nginx Ingress detects endpoint removal via informer and updates upstream configuration. In-flight requests on existing connections continue to terminating pod. nginx.ingress.kubernetes.io/proxy-next-upstream: "non_idempotent error timeout" retries failed requests. Service spec.sessionAffinity with terminationGracePeriodSeconds ensures proper draining.

Q49. What is Contour/Envoy as an Ingress alternative?

Contour uses Envoy proxy (instead of Nginx) as data plane. HTTPProxy CRD (Contour-specific) provides richer routing than Ingress. Advantages: Envoy hot config reload (no process restart), better gRPC support, Envoy's advanced LB algorithms, circuit breaking built-in. Gateway API support: Contour is a Gateway API-conformant implementation. Used by VMware Tanzu, preferred for Envoy-native service mesh environments.

Q50. How do you set up wildcard certificate with DNS-01 challenge?

ClusterIssuer with dns01 solver: configure Route53/Cloudflare credentials (via Secret). Certificate CR: spec.dnsNames: ["*.example.com", "example.com"]. cert-manager creates TXT record _acme-challenge.example.com in Route53. ACME server verifies TXT record and issues wildcard cert. Single cert covers all subdomains. Mount in Ingress TLS section or as default cert.

Q51. How does Nginx Ingress support custom error pages?

Deploy custom-error-pages Service that serves branded HTML error pages at /. ConfigMap: custom-http-errors: 404,503,502 - list of codes to intercept. default-backend-service: custom-error-pages Service. When backend returns 503, Nginx proxies request to error-pages/503. Error page service receives: X-Code, X-Format, X-Original-URI headers for context.

Q52. What is load balancing with upstream hashing?

nginx.ingress.kubernetes.io/upstream-hash-by: $request_uri - consistent hashing by URL. Routes same URL to same pod (useful for caching at app level). $cookie_SESSIONID - hash by session cookie (sticky without nginx affinity cookie). $remote_addr - hash by client IP (same client always hits same pod). upstream-hash-by-subset: use subset of upstreams for consistent hash subset LB.

Q53. How do you troubleshoot cert-manager certificate not becoming Ready?

kubectl describe certificate cert-name: check status.conditions for error messages. kubectl describe certificaterequest: shows ACME order details. kubectl describe challenge: shows HTTP-01/DNS-01 challenge status. Common issues: HTTP-01 challenge URL not reachable (firewall/CDN blocking port 80), DNS-01 TXT record not propagating (DNS TTL too high), ACME rate limits exceeded (use Let's Encrypt staging for testing).